



南開大學
Nankai University

计算机学院
数据安全课程实验报告

秘密共享实践

姓名：孔祥昊

2311439

专业：计算机科学与技术

2026 年 4 月 30 日

目录

1 实验要求	2
2 实验原理	2
2.1 核心定义与目标	2
2.2 构造阶段（秘密拆分）	2
2.3 重构阶段（秘密恢复）	2
2.4 正确性证明	3
2.5 安全性证明	3
3 实验	3
3.1 ss_function.py: 基础数学函数库（模运算、多项式生成、重构）	3
3.2 participant_share.py: 参与者输入数据并分发秘密份额	4
3.3 participant_aggregate.py: 参与者对收到的份额进行本地求和	5
3.4 average_collector.py: 收集聚合份额，重构总和并计算平均值	5
4 代码运行及结果验证	6
4.1 Step 1: 分发秘密并交换份额	6
4.2 Step 2: 本地聚合	7
4.3 Step 3: 计算结果	8
5 Github 链接	8

1 实验要求

- 借鉴实验 3.2，实现三个人对于他们拥有的数据的平均值的计算。

2 实验原理

秘密共享 (Secret Sharing) 是现代密码学中的一项核心技术，旨在解决秘密数据的分布式存储与管理问题。其中最著名的方案是由 Adi Shamir 在 1979 年提出的 Shamir (k, n) 门限秘密共享方案。

2.1 核心定义与目标

- **方案定义**：在一个 (k, n) 门限方案中，秘密 S 被拆分为 n 个份额 (Shares)，分发给 n 个参与者。
- **存取结构**：
 - **正确性**：任意不少于 k 个参与者通过协作，可以精确地重构出秘密 S 。
 - **安全性**：任意少于 k 个参与者即使拥有其持有的份额，也无法获得关于秘密 S 的任何信息 (即达到信息论安全)。

2.2 构造阶段 (秘密拆分)

- **初始化**：秘密分发者随机选取 $k - 1$ 个元素 $a_1, a_2, \dots, a_{k-1} \in \mathbb{Z}_p$ 。
- **构造多项式**：定义一个 $k - 1$ 次多项式：

$$f(x) = S + a_1x + a_2x^2 + \dots + a_{k-1}x^{k-1} \pmod{p}$$

这里， $f(0) = S$ 即为我们保护的秘密。

- **份额生成**：分发者选取 n 个互不相同的非零 x 值 (通常取 $x_i = i$)，计算并将份额 $D_i = (i, f(i))$ 私密地发送给第 i 个参与者。

2.3 重构阶段 (秘密恢复)

- **数据收集**：假设有 k 个参与者通过协作，收集到了点集 $\{(x_1, y_1), (x_2, y_2), \dots, (x_k, y_k)\}$ 。
- **拉格朗日插值法**：利用拉格朗日插值公式，可以重构出唯一的 $k - 1$ 次多项式：

$$f(x) = \sum_{j=1}^k y_j \cdot L_j(x) \pmod{p}$$

其中 $L_j(x)$ 为插值基函数： $L_j(x) = \prod_{m=1, m \neq j}^k \frac{x - x_m}{x_j - x_m} \pmod{p}$ 。

- **求解秘密**：由于 $S = f(0)$ ，代入 $x = 0$ 得到：

$$S = \sum_{j=1}^k y_j \left(\prod_{m=1, m \neq j}^k \frac{-x_m}{x_j - x_m} \right) \pmod{p}$$

2.4 正确性证明

- **逻辑**：根据拉格朗日插值定理，给定 k 个不同的点，存在且仅存在一个次数小于 k 的多项式通过这些点。
- **证明**：由于分发阶段构造的 $f(x)$ 次数为 $k-1$ ，且这 k 个点均在 $f(x)$ 上，因此通过插值法重构出的多项式必然是原多项式 $f(x)$ 。代入 $x=0$ 得到的常数项必然等于初始秘密 S 。

2.5 安全性证明

- **逻辑**：证明当拥有 $k-1$ 个点时，秘密 S 的先验概率等于后验概率。
- **数学推导**：
 - 假设攻击者持有 $k-1$ 个点。对于有限域 $\mathbb{Z}_p$ 中任何一个可能的秘密 $S' \in [0, p-1]$ ，都存在唯一的一个系数序列 $(a'_1, a'_2, \dots, a'_{k-1})$ 使得构造的多项式 $g(x)$ 既通过这 $k-1$ 个已知点，且满足 $g(0) = S'$ 。
 - 换言之，在已知 $k-1$ 个点的情况下，候选秘密 S' 与 $k-1$ 次多项式的系数序列是一一对应的。
 - 由于分发者在初始化时是从 p^{k-1} 种可能的系数组合中等概率随机选取的，因此对于这 $k-1$ 个已知点，每一个可能的秘密 S' 对应的概率均为 $1/p$ 。
 - **结论**：持有少于 k 个份额无法获得关于 S 的任何统计学优势，方案是信息论安全的。

3 实验

为完成本次实验，我们将编写如下四个 python 代码。

3.1 ss_function.py：基础数学函数库（模运算、多项式生成、重构）

```
1 import random
2
3 def quickpower(a, b, p):
4     a = a % p
5     ans = 1
6     while b != 0:
7         if b & 1: ans = (ans * a) % p
8         b >>= 1
9         a = (a * a) % p
10    return ans
11
12 def get_polynomial(x0, T, p, fname):
13     f = [x0]
14     for i in range(T):
15         f.append(random.randrange(0, p))
16    return f
17
18 def count_polynomial(f, x, p):
```

```

19     ans = f[0]
20     for i in range(1, len(f)):
21         ans = (ans + f[i] * quickpower(x, i, p)) % p
22     return ans
23
24 def restructure_polynomial(x, fx, t, p):
25     ans = 0
26     for i in range(t):
27         fx[i] = fx[i] % p
28         fxi = 1
29         for j in range(t):
30             if j != i:
31                 # 模块化逆元计算
32                 fxi = (-1 * fxi * x[j] * quickpower(x[i] - x[j], p - 2, p)) % p
33         fxi = (fxi * fx[i]) % p
34         ans = (ans + fxi) % p
35     return ans % p

```

- **核心功能：**该模块实现了 Shamir 秘密共享方案在有限域 $\mathbb{Z}_p$ 上的所有核心数学运算。
- **数学原理：**
 - **快速幂运算：**通过 `quickpower` 函数实现 $a^b \pmod{p}$, 将幂运算的时间复杂度优化至 $O(\log b)$, 用于后续的模逆元计算。
 - **多项式构造：**`get_polynomial` 用于生成一个 T 次随机多项式 $f(x) = a_0 + a_1x + \dots + a_Tx^T \pmod{p}$, 其中常数项 a_0 为秘密值 s 。本实验采用 $(2, 3)$ 门限, 故 $T = 1$ 。
 - **拉格朗日插值重构：**`restructure_polynomial` 函数基于拉格朗日插值法, 利用 t 个份额点 (x_i, y_i) 重构多项式并求解 $f(0)$ 。在模运算环境下, 除法通过乘以模逆元 (即 $b^{p-2} \pmod{p}$) 实现。

3.2 participant_share.py: 参与者输入数据并分发秘密份额

```

1 import ss_function as ss_f
2
3 p = 1000000007
4 user_id = int(input("请输入参与者 ID (1-3): "))
5 secret = int(input(f"请输入参与者 {user_id} 的私有数值: "))
6
7 # (2,3) 门限: 多项式次数为 1 (t-1)
8 shares_x = [1, 2, 3]
9 f = ss_f.get_polynomial(secret, 1, p, str(user_id))
10
11 for i in range(1, 4):
12     share_y = ss_f.count_polynomial(f, shares_x[i-1], p)
13     with open(f'share_{user_id}_to_{i}.txt', 'w') as f_out:
14         f_out.write(str(share_y))
15 print(f"参与者 {user_id} 的份额分发完毕。")

```

- **逻辑流程**: 该脚本模拟参与者的输入阶段。每个参与者输入其私有的数值 s_i , 系统为其生成一个独立的随机一次多项式 $f_i(x)$, 满足 $f_i(0) = s_i$ 。
- **份额生成**: 程序为每位参与者计算三个坐标点的数据: $f_i(1), f_i(2), f_i(3)$ 。
- **数据传输模拟**: 生成的份额被保存为特定的文件名 (如 `share_1_to_2.txt`), 以此模拟秘密份额从参与者 i 分发至参与者 j 的过程。由于多项式的系数是随机的, 单份份额不会泄露任何原始输入信息。

3.3 participant_aggregate.py: 参与者对收到的份额进行本地求和

```

1 p = 10000000007
2 user_id = int(input("请输入参与者 ID (1-3) 进行聚合: "))
3
4 local_sum = 0
5 for i in range(1, 4):
6     with open(f'share_{i}_to_{user_id}.txt', 'r') as f:
7         local_sum = (local_sum + int(f.read())) % p
8
9 with open(f'aggregated_share_{user_id}.txt', 'w') as f_out:
10     f_out.write(str(local_sum))
11 print(f"参与者 {user_id} 本地份额聚合完毕。")

```

- **加法同态原理**: 这是多方安全计算 (MPC) 的核心。由于 Shamir 秘密共享具有加法同态性, 若令总和多项式为 $F(x) = \sum f_i(x)$, 则有 $F(0) = \sum f_i(0) = \sum s_i$ 。
- **本地运算逻辑**: 每个参与者 j 收集所有分发给自己的份额 $f_i(j)$, 并在本地执行求和运算: $D_j = \sum_{i=1}^3 f_i(j) \pmod{p}$ 。
- **隐私保护**: 此阶段参与者仅得到“份额之和”, 依然无法反推其他人的具体原始数据, 确保了计算过程的隐私性。

3.4 average_collector.py: 收集聚合份额, 重构总和并计算平均值

```

1 import ss_function as ss_f
2
3 p = 10000000007
4 # 选取参与者 1 和 2 的聚合结果进行重构
5 ids = [1, 2]
6 collected_shares = []
7
8 for i in ids:
9     with open(f'aggregated_share_{i}.txt', 'r') as f:
10         collected_shares.append(int(f.read()))
11
12 # 1. 重构总和
13 total_sum = ss_f.restructure_polynomial(ids, collected_shares, 2, p)

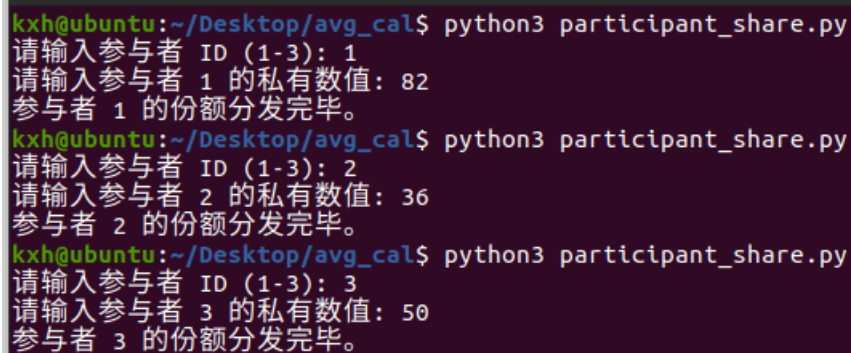
```

```
14
15 # 2. 计算平均值
16 # 注意：在有限域内除法需用逆元，但实验演示通常直接在实数域除以 3 更直观
17 average = total_sum / 3
18
19 print(f"--- 实验结果 ---")
20 print(f"重构出的数据总和为: {total_sum}")
21 print(f"三人的数据平均值为: {average:.2f}")
```

- **门限触发**：作为结果收集方，该脚本仅需读取任意两个参与者的聚合份额（本实验取 ID 为 1 和 2 的份额），即可通过 (2,3) 门限重构总和。
- **数据还原**：调用重构函数计算 $F(0)$ ，得到三个原始数据的代数和 $Total_Sum = (s_1 + s_2 + s_3) \pmod{p}$ 。
- **均值输出**：在重构出总和后，程序通过执行 $Average = Total_Sum/3$ 输出最终的平均值。这一步完成了从加密态计算到明文态结果的转化。

4 代码运行及结果验证

4.1 Step 1: 分发秘密并交换份额



```
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 1
请输入参与者 1 的私有数值: 82
参与者 1 的份额分发完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 2
请输入参与者 2 的私有数值: 36
参与者 2 的份额分发完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 3
请输入参与者 3 的私有数值: 50
参与者 3 的份额分发完毕。
```

图 4.1: 三个人分发秘密

执行如图4.1中的指令后，我们会得到 9 个 share_i_to_j.txt 文件，如下所示。

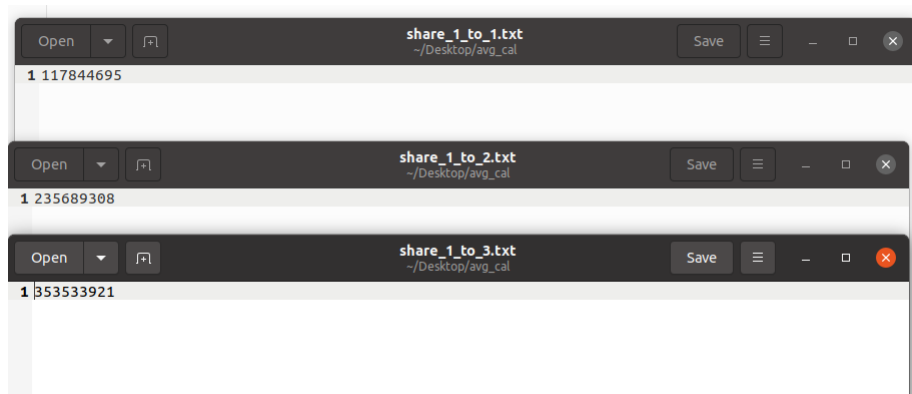


图 4.2: 第一个人

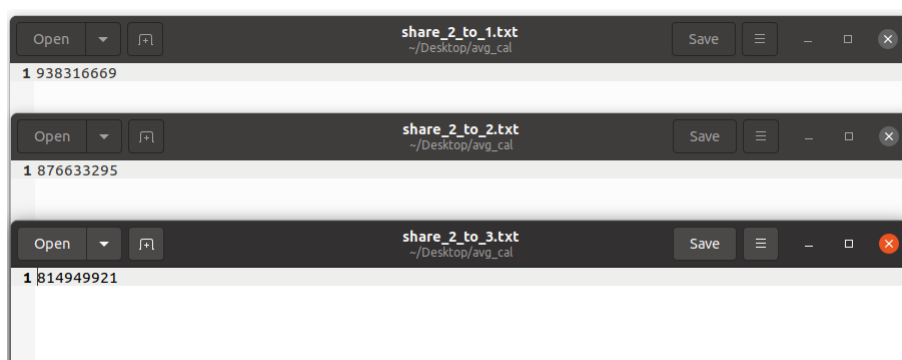


图 4.3: 第二个人

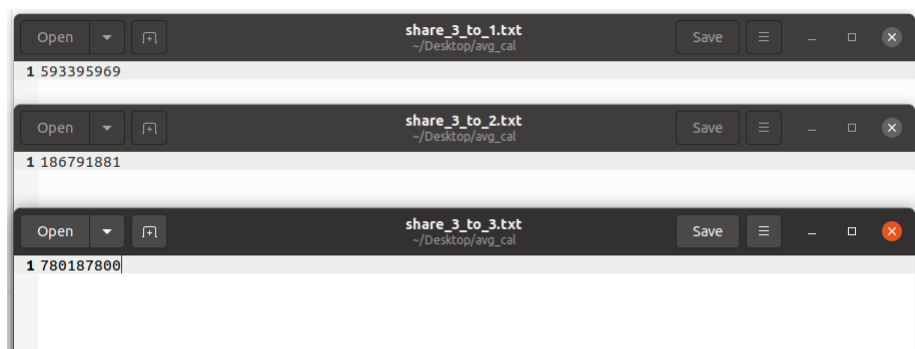


图 4.4: 第三个人

4.2 Step 2: 本地聚合

三个参与者分别执行 `participant_aggregate.py`，生成 3 个 `aggregated_share_i.txt` 文件。

```
1 python3 participant_aggregate.py
2 1
3 python3 participant_aggregate.py
4 2
5 python3 participant_aggregate.py
6 3
```


执行上述指令，我们会得到如下所示文件。

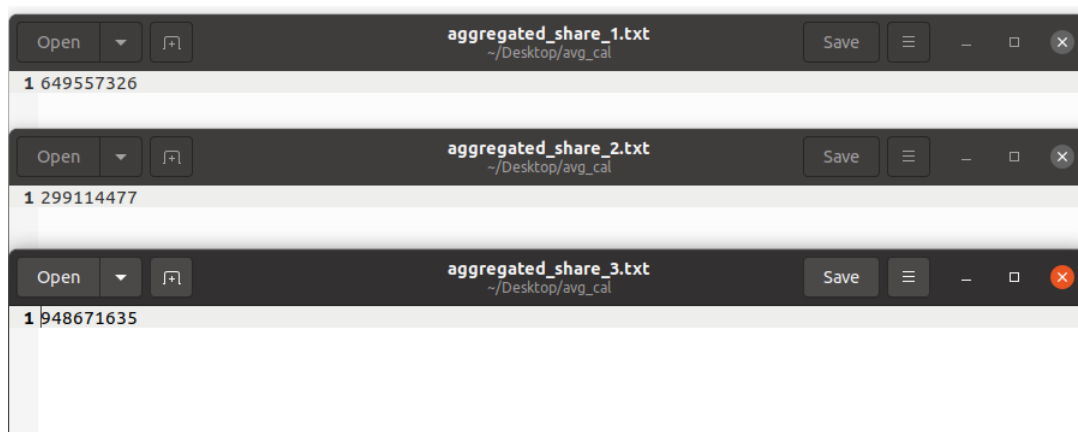


图 4.5: 聚合结果

4.3 Step 3: 计算结果

输入命令 `python3 average_collector.py`，得到如下结果。

```
kxh@ubuntu: ~/Desktop/avg_cal
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 1
请输入参与者 1 的私有数值: 82
参与者 1 的份额分发完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 2
请输入参与者 2 的私有数值: 36
参与者 2 的份额分发完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_share.py
请输入参与者 ID (1-3): 3
请输入参与者 3 的私有数值: 50
参与者 3 的份额分发完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_aggregate.py
请输入参与者 ID (1-3) 进行聚合: 1
参与者 1 本地份额聚合完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_aggregate.py
请输入参与者 ID (1-3) 进行聚合: 2
参与者 2 本地份额聚合完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 participant_aggregate.py
请输入参与者 ID (1-3) 进行聚合: 3
参与者 3 本地份额聚合完毕。
kxh@ubuntu:~/Desktop/avg_cal$ python3 average_collector.py
--- 实验结果 ---
重构出的数据总和为: 168
三人的数据平均值为: 56.00
kxh@ubuntu:~/Desktop/avg_cal$
```

图 4.6: 计算平均值

如上图所示，如我们所期望的，三个人的平均值被正确计算。

5 Github 链接

访问[Github 链接](#)获取代码。